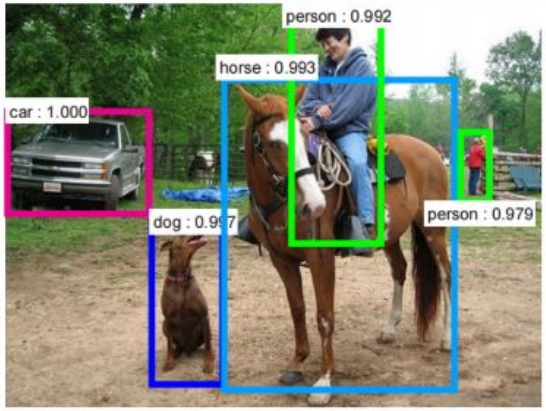
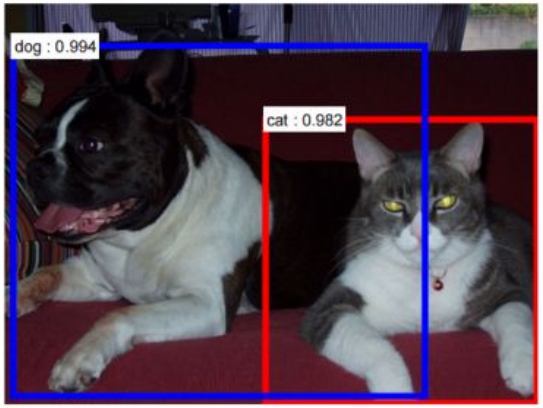
Deep Learning for Computer Vision

JENNIFER JOYCE M. MONTEMAYOR - MAULANA

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology





Computer Vision is the earliest and biggest success story of deep learning.

Everyday you are interacting with deep learning models - camera filters, video filters, Optical Character Recognition (OCR) softwares, Google image search, and many more.

Deep learning models is the heart of cutting-edge research in autonomous driving, robotics, facial recognition, AI-assisted medical diagnosis, agricultural monitoring, manufacturing fault detection.



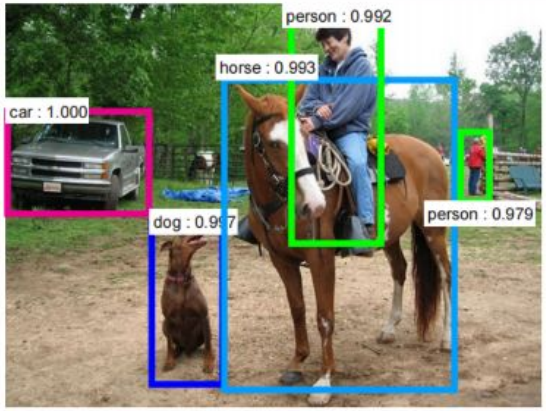
a soccer player is kicking a soccer ball

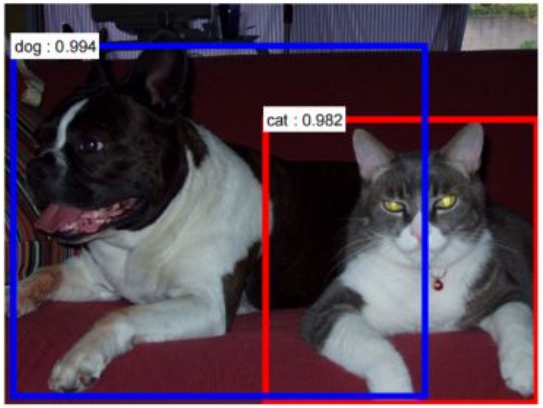


a street sign on a pole in front of a building



a couple of giraffe standing next to each other







a soccer player is kicking a soccer ball



a street sign on a pole in front of a building



a couple of giraffe standing next to each other

Convolutional Neural Networks

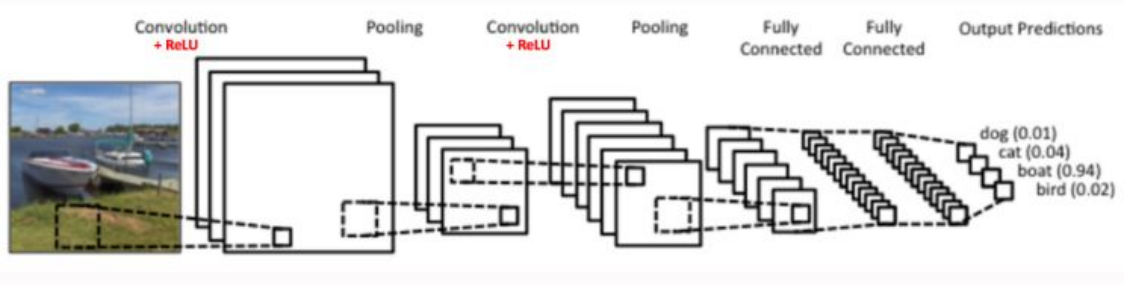
- ConvNet or CNN
- a type of deep learning architecture / category of neural networks that have proven to be effective in image recognition and classification

Figure 1 shows a ConvNet being used to recognize everyday objects, humans, and animals.

Figure 2 shows a ConvNet being able to recognize scenes and the system is able to suggest relevant captions

The LeNet Architecture (1990s)

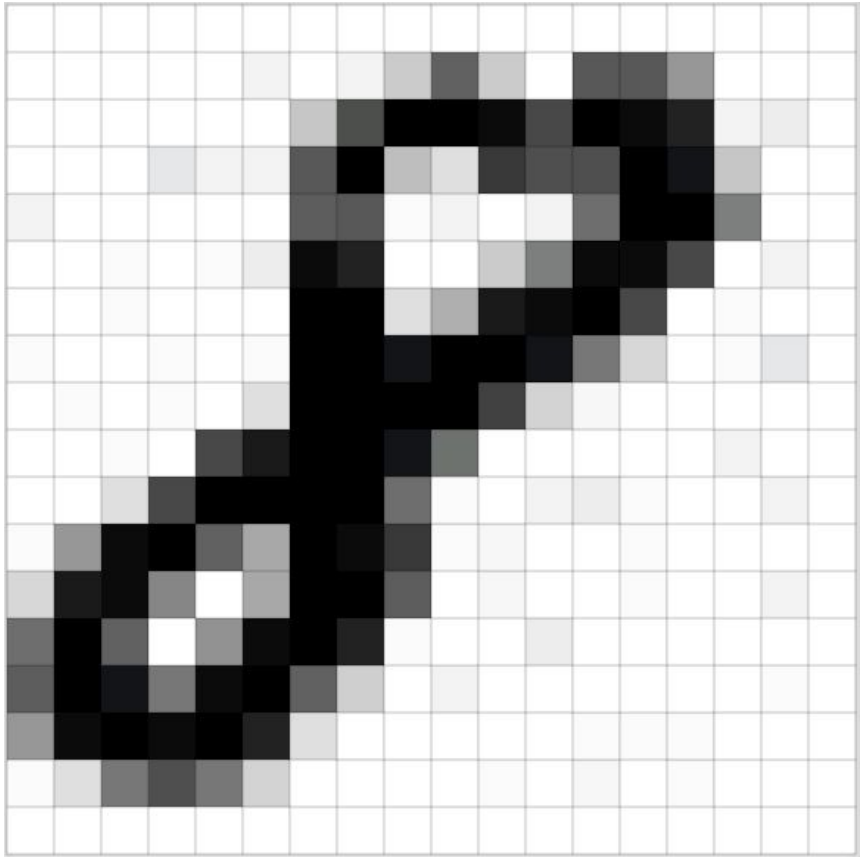
- In 1990, computer scientist Yann André LeCun implemented Convolutional Neural Networks (CNN) in their paper "*Gradient-Based Learning Applied to Document Recognition*"
- There have several new architectures proposed in recent years which are improvements over LeNet



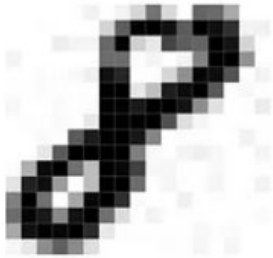
The following operations are basic building blocks of every CNN:

1. Convolution
2. Non Linearity
3. Pooling or Subsampling or Downsampling
4. Classification

To a computer an image is really a grid of numbers that represent how dark each pixel is.



To feed an image into a neural network, we treat the 18x18 pixel image as an array of 324 numbers.



=

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 12, 0, 11, 39, 137, 37, 0, 152, 147, 84, 0, 0, 0, 0,
0, 1, 0, 0, 0, 41, 160, 250, 255, 235, 162, 255, 238, 206, 11, 13, 0, 0, 0, 0, 16, 9, 9, 150, 251, 45, 21, 184, 159, 154, 2
55, 233, 40, 0, 0, 10, 0, 0, 0, 0, 0, 145, 146, 3, 10, 0, 11, 124, 253, 255, 107, 0, 0, 0, 0, 3, 0, 4, 15, 236, 216, 0, 0,
38, 109, 247, 240, 169, 0, 11, 0, 1, 0, 2, 0, 0, 0, 253, 253, 23, 62, 224, 241, 255, 164, 0, 5, 0, 0, 6, 0, 0, 4, 0, 3, 252
, 250, 228, 255, 255, 234, 112, 28, 0, 2, 17, 0, 0, 2, 1, 4, 0, 21, 255, 253, 251, 255, 172, 31, 8, 0, 1, 0, 0, 0, 0, 0, 4,
 0, 163, 225, 251, 255, 229, 120, 0, 0, 0, 0, 0, 11, 0, 0, 0, 0, 21, 162, 255, 255, 254, 255, 126, 6, 0, 10, 14, 6, 0, 0, 9
, 0, 3, 79, 242, 255, 141, 66, 255, 245, 189, 7, 8, 0, 0, 5, 0, 0, 0, 0, 26, 221, 237, 98, 0, 67, 251, 255, 144, 0, 8, 0, 0
, 7, 0, 0, 11, 0, 125, 255, 141, 0, 87, 244, 255, 208, 3, 0, 0, 13, 0, 1, 0, 1, 0, 0, 145, 248, 228, 116, 235, 255, 141, 34
, 0, 11, 0, 1, 0, 0, 0, 1, 3, 0, 85, 237, 253, 246, 255, 210, 21, 1, 0, 1, 0, 0, 6, 2, 4, 0, 0, 0, 6, 23, 112, 157, 114, 32
, 0, 0, 0, 0, 2, 0, 8, 0, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

When we flatten our image, we lose spatial information that is unique and relevant to the image.

It also makes the neural network very large and computation expensive.

Convolution

ConvNets derive their names from the “convolution” operator.

The primary purpose of Convolution in a ConvNet is to extract features from the input image to preserve the spatial relationship between pixels by learning image features using small squares of input data.

Consider a 5x5 input image and another 3x3 matrix,

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

(note that for a grayscale image, pixel values range from 0 to 255, the green matrix below is a special case where pixel values are only 0 and 1)

Convolution

ConvNets derive their names from the “convolution” operator.

The primary purpose of Convolution in a ConvNet is to extract features from the input image to preserve the spatial relationship between pixels by learning image features using small squares of input data.

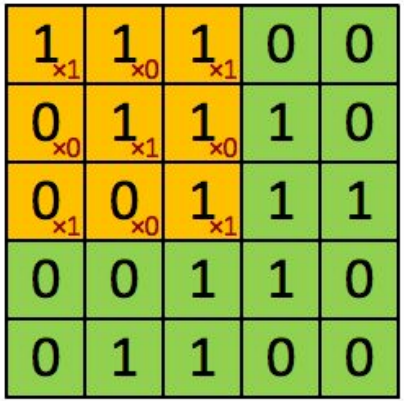
Consider a 5x5 input image and another 3x3 matrix,

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

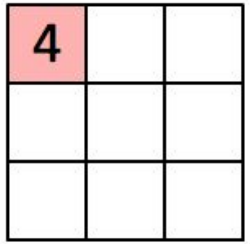
1	0	1
0	1	0
1	0	1

(note that for a grayscale image, pixel values range from 0 to 255, the green matrix below is a special case where pixel values are only 0 and 1)

CONVOLUTION OPERATION



Image



Convolved Feature

Convolution

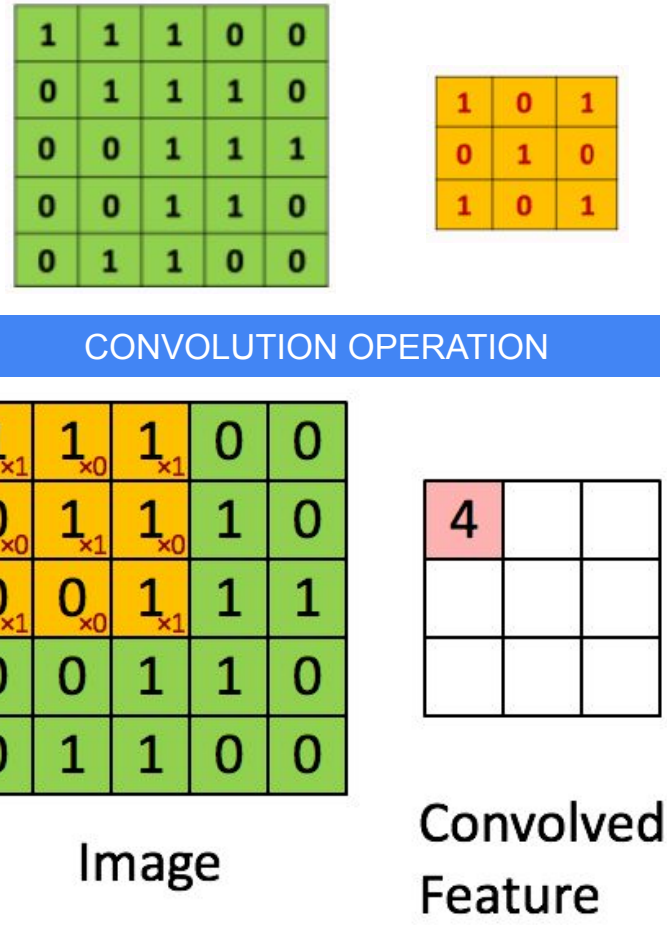
Slide the orange matrix called a “**filter**”/ “**kernel**”/ “**feature detector**” over the original matrix by 1 pixel. This is movement is called a “**stride**”.

For every position, we compute:

Element wise multiplication between the two matrices.

Add the outputs to get the final integer which forms a single element on the output matrix called a “**convolved feature**” / “**activation map**” / “**feature map**”.

Different values of the filter matrix will produce different feature maps for the same input image.



Convolution



Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or **edge detection**	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

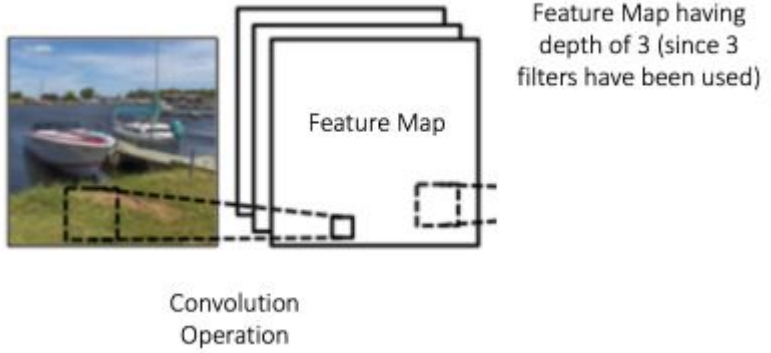
Convolution



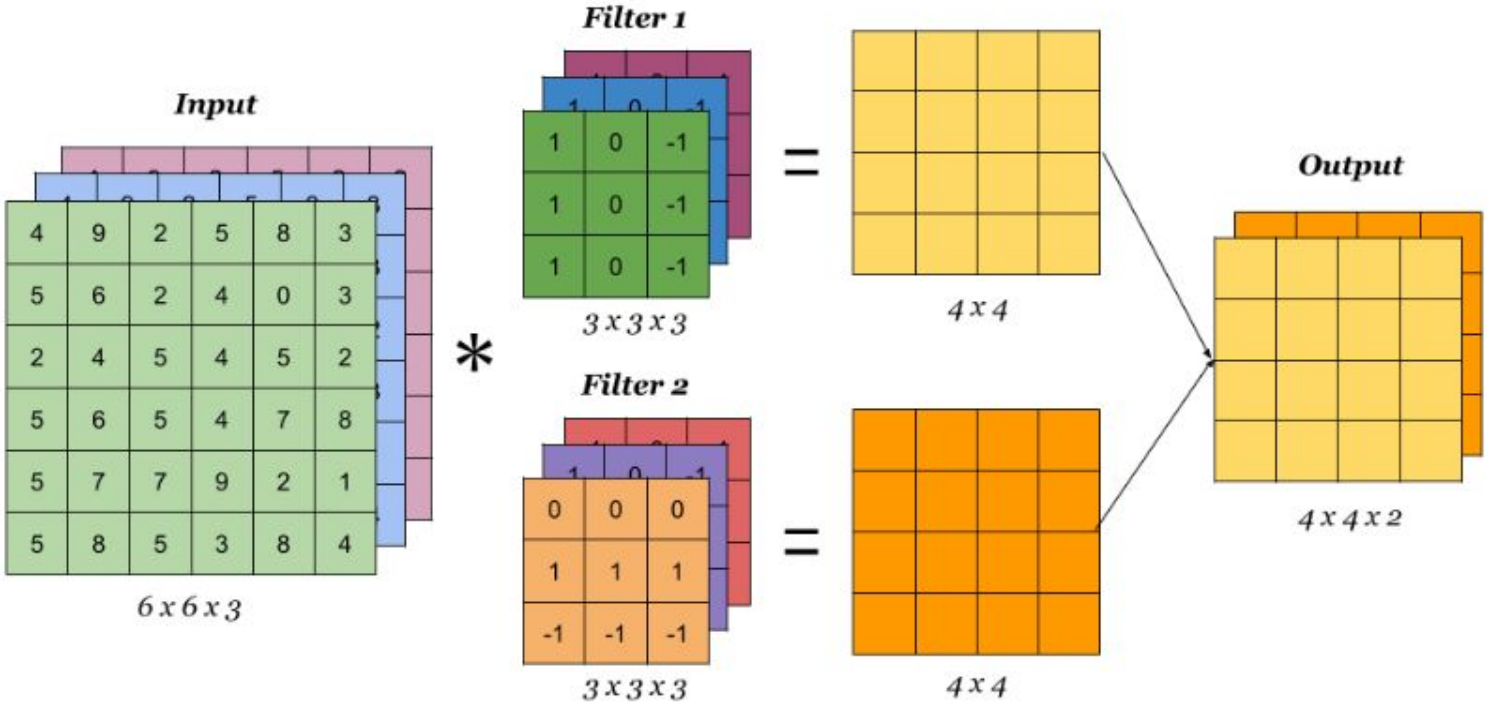
Convolution

The size of the **Feature Map** or the **Convolved Feature** is controlled by 3 parameters:

1. Depth
   - Number of filters for the convolution operation where each filter detects different patterns or features in the input
   - Essential because it allows a convolutional layer to capture and represent various features and patterns
   - Affects the computational complexity of the network
   - Different filters can specialize in detecting edges, textures, or more complex patterns, ultimately contributing to the network's ability to understand and classify visual information



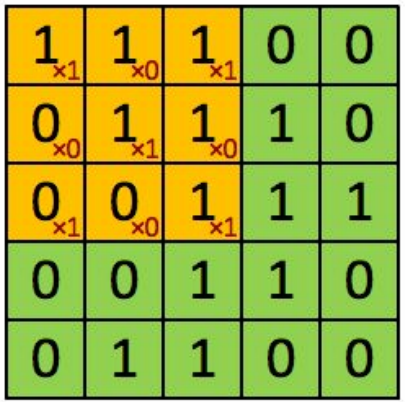
Convolution



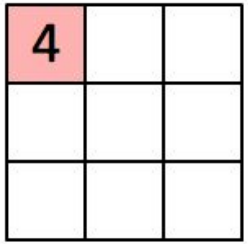
Convolution

The size of the **Feature Map** or the **Convolved Feature** is controlled by 3 parameters:

2. Stride
   - Number of pixels by which we slide our filter matrix over the input matrix
     - Stride = 1, move filter 1 pixel at a time
     - Stride = 2, filters move 2 pixels at a time
   - Step size or how much the filter is moved across (shifts horizontally and vertically) at each step during the convolution operation
   - Larger slides produce smaller feature maps, downsample the input feature map
     - Stride = 2 will have half the width and half the height of the input feature map

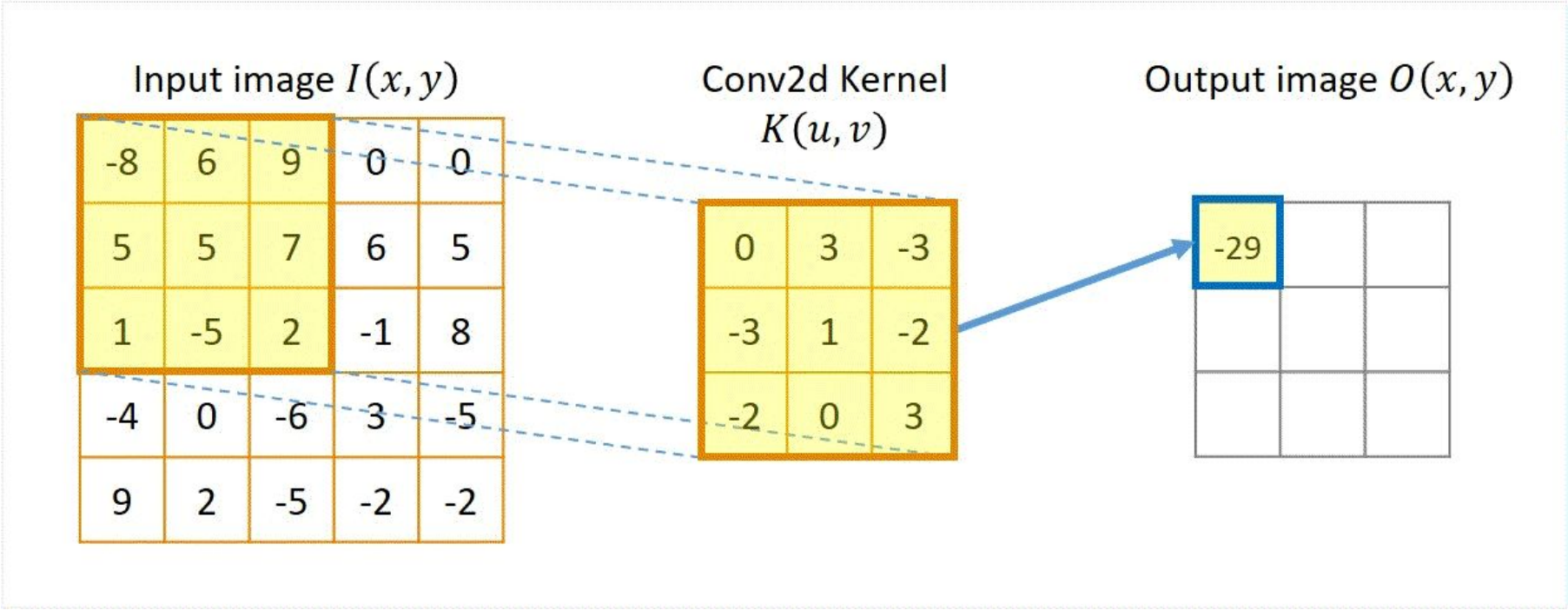


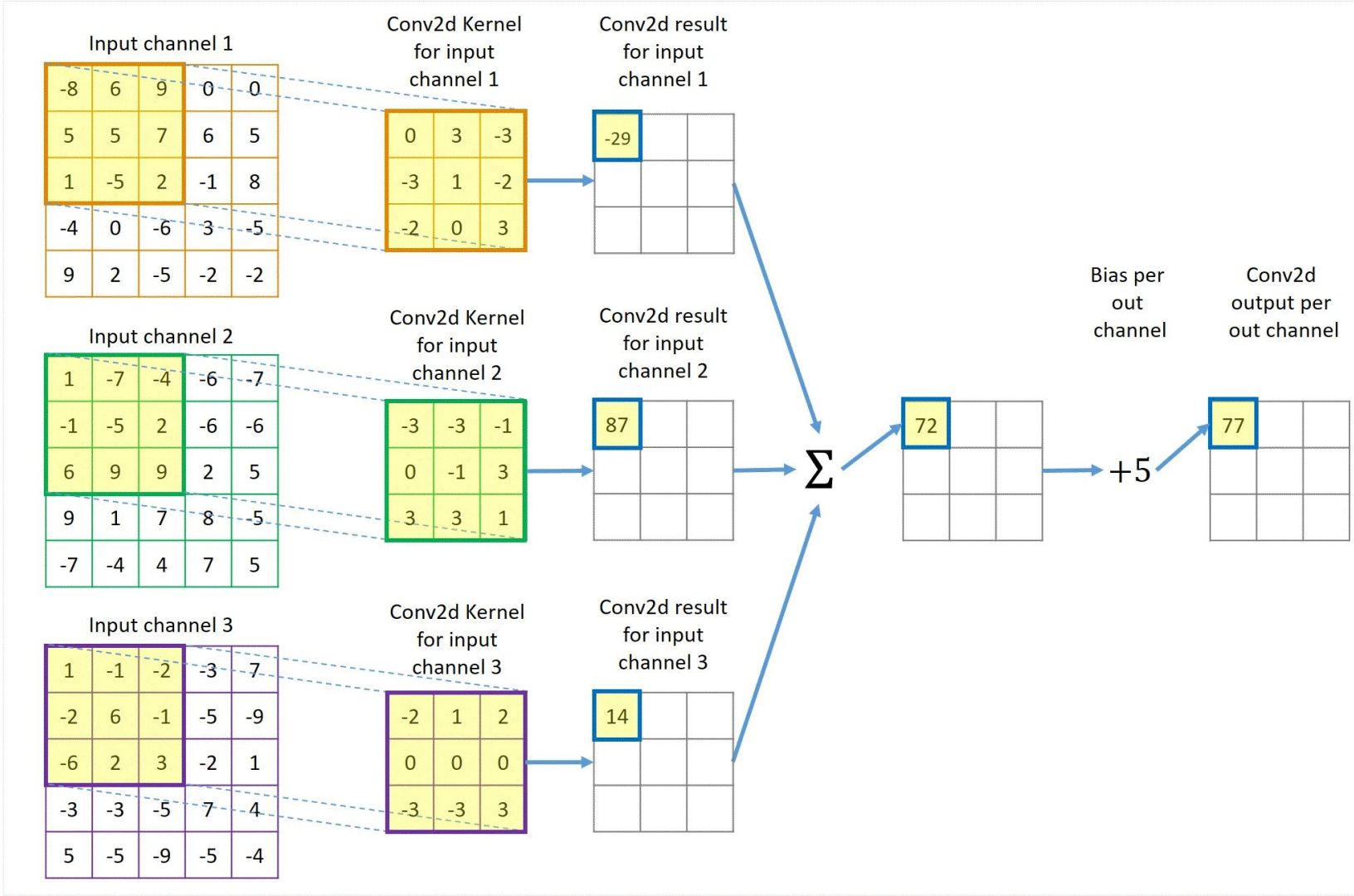
Image



Convolved Feature

Convolution

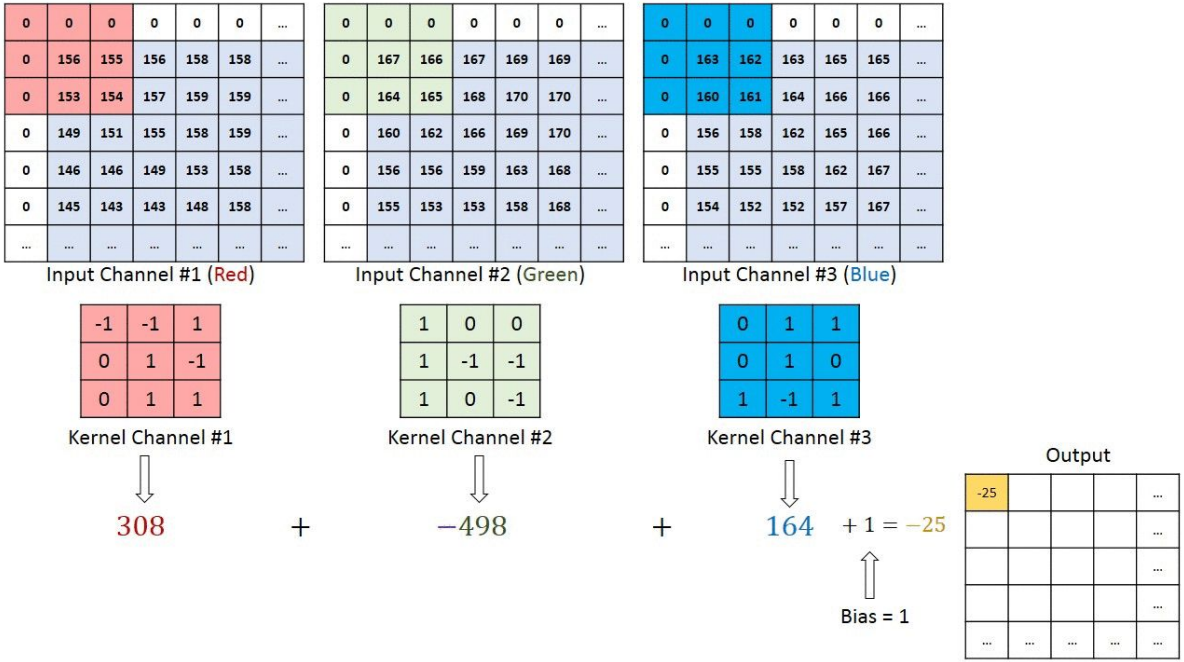


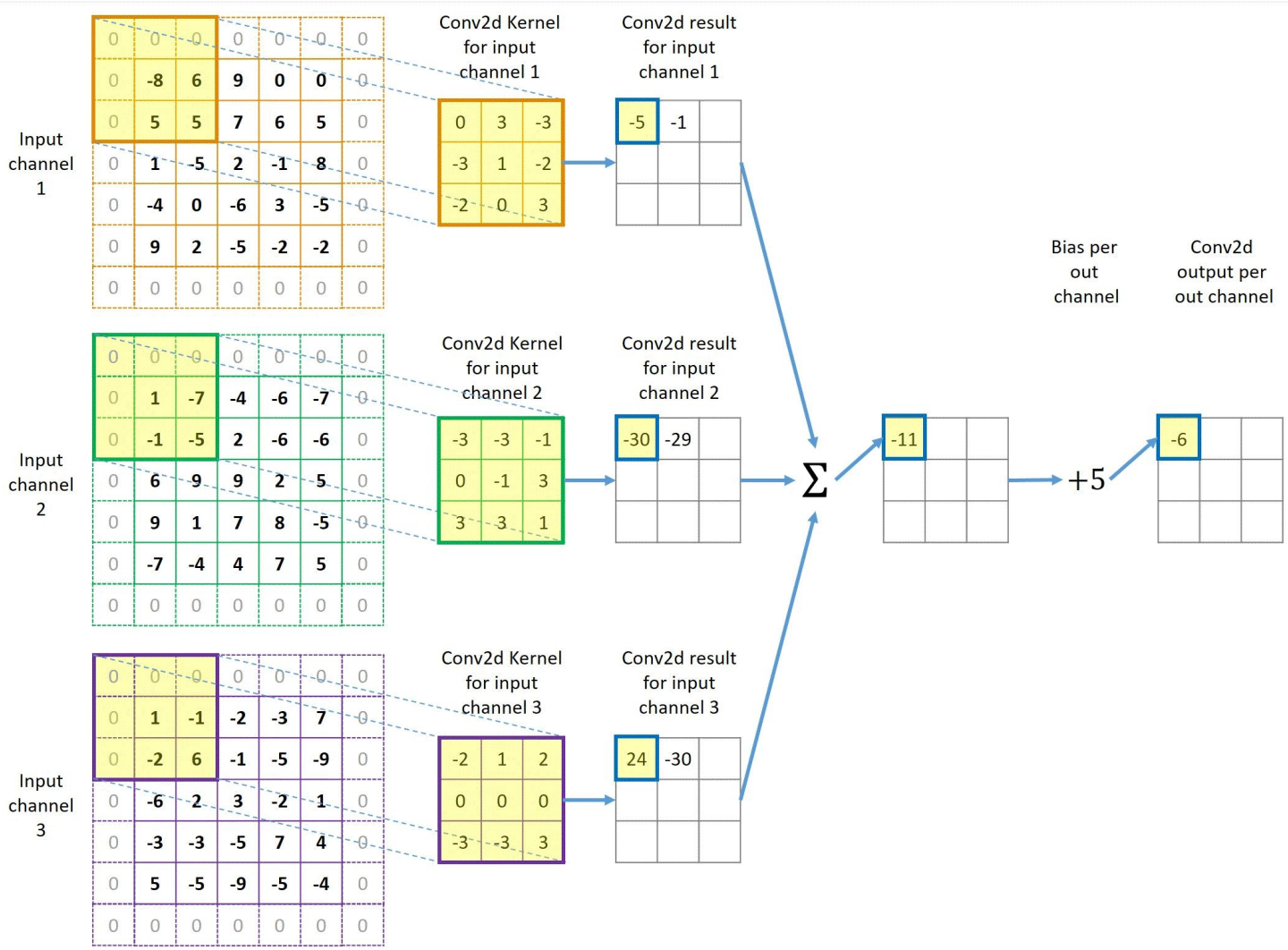

Input channel 1
-8 6 9 0 0
5 5 7 6 5
1 -5 2 -1 8
-4 0 -6 3 -5
9 2 -5 -2 -2
Conv2d Kernel for input channel 1
0 3 -3
-3 1 -2
-2 0 3
Conv2d result for input channel 1
-29
Input channel 2
1 -7 -4 -6 -7
-1 -5 2 -6 -6
6 9 9 2 5
9 1 7 8 -5
-7 -4 4 7 5
Conv2d Kernel for input channel 2
-3 -3 -1
0 -1 3
3 3 1
Conv2d result for input channel 2
87
Input channel 3
1 -1 -2 -3 7
-2 6 -1 -5 -9
-6 2 3 -2 1
-3 -3 -5 7 4
5 -5 -9 -5 -4
Conv2d Kernel for input channel 3
-2 1 2
0 0 0
-3 -3 3
Conv2d result for input channel 3
14
Σ
72
Bias per out channel
+5
Conv2d output per out channel
77

Convolution

The size of the **Feature Map** or the **Convolved Feature** is controlled by 3 parameters:

3. Zero-padding
   - Used to adjust the spatial dimensions of the input data or feature map before applying convolution
   - Involves adding rows and columns of zeros to the edges of the input matrix (increasing the dimensions of the input)
   - Ensures that the filter remains centered to avoid losing information at the edges



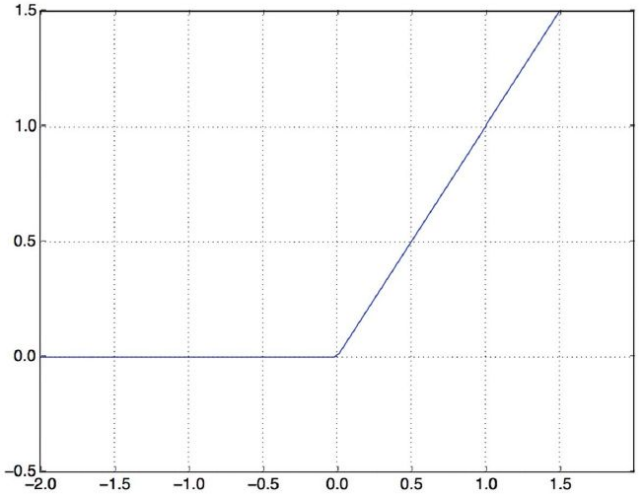

Input channel 1
Input channel 2
Input channel 3
Conv2d Kernel for input channel 1
Conv2d Kernel for input channel 2
Conv2d Kernel for input channel 3
Conv2d result for input channel 1
Conv2d result for input channel 2
Conv2d result for input channel 3
Bias per out channel
Conv2d output per out channel
+5

Non Linearity

The choice of activation function often depends on the specific problem and testing to determine which one works best for the task at hand.

ReLU (Rectified Linear Unit)

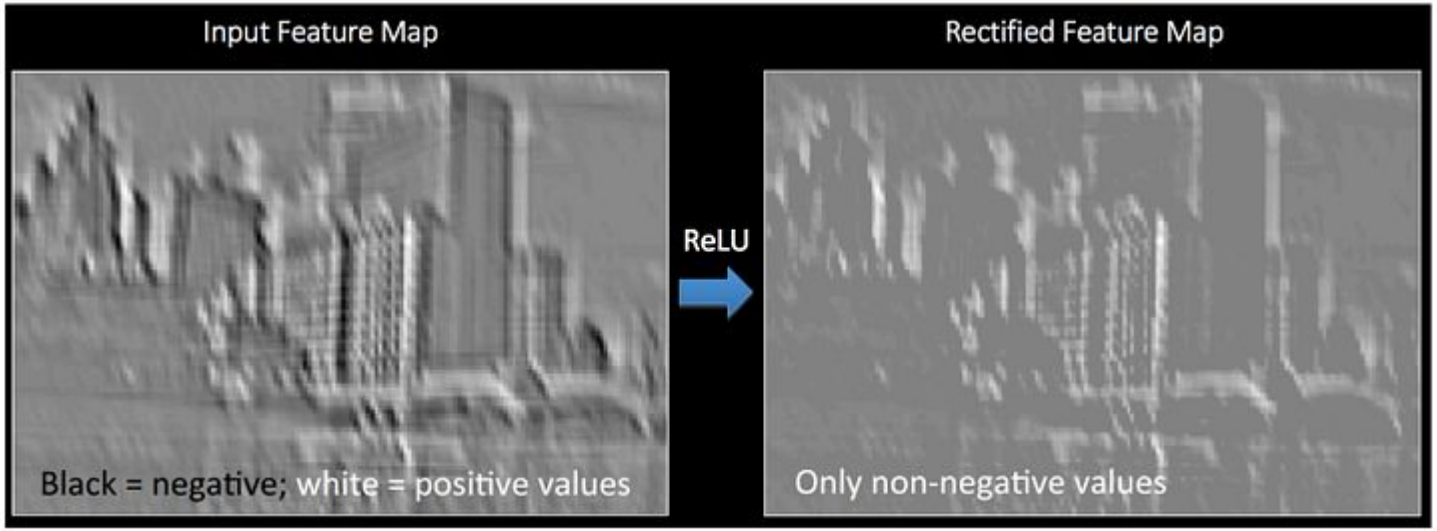
- Computationally cheap to compute `f(x) = max (0, x)`
- Less susceptible to vanishing gradient problem
  - Happens when gradients in the network become extremely small leading to slow convergence or difficulty in training deep networks
  - Two possible gradients:
    - Positive gradient (1) - when input is greater than 0, the gradient is a constant 1
    - Zero gradient (0) - when the input is less than or equal to 0, the gradient is a constant 0 (making the neuron inactive when the input is negative)
- Can suffer in “dying ReLU” problem where neurons can get stuck in an inactive state during training
  - Leaky ReLU, Parametric ReLU



Non Linearity

The choice of activation function often depends on the specific problem and testing to determine which one works best for the task at hand.

ReLU (Rectified Linear Unit)

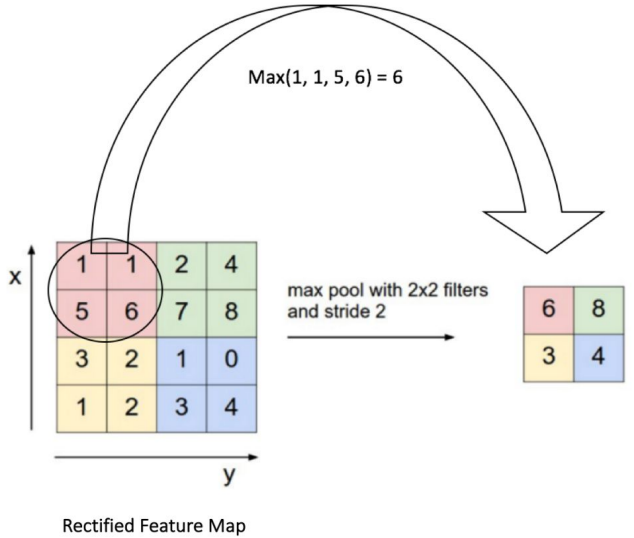


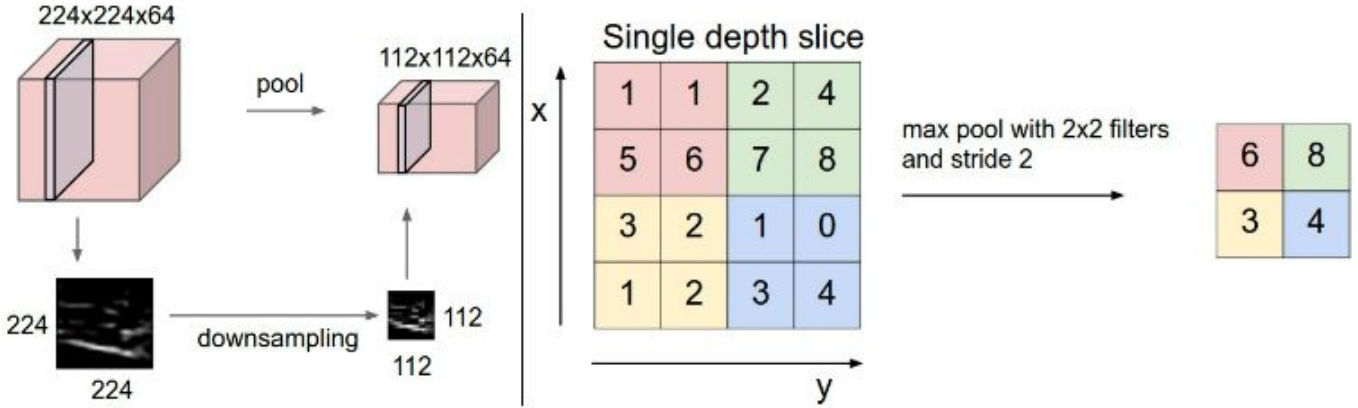
Pooling

It is common to periodically insert a pooling layer between successive convolutional layers to progressively:

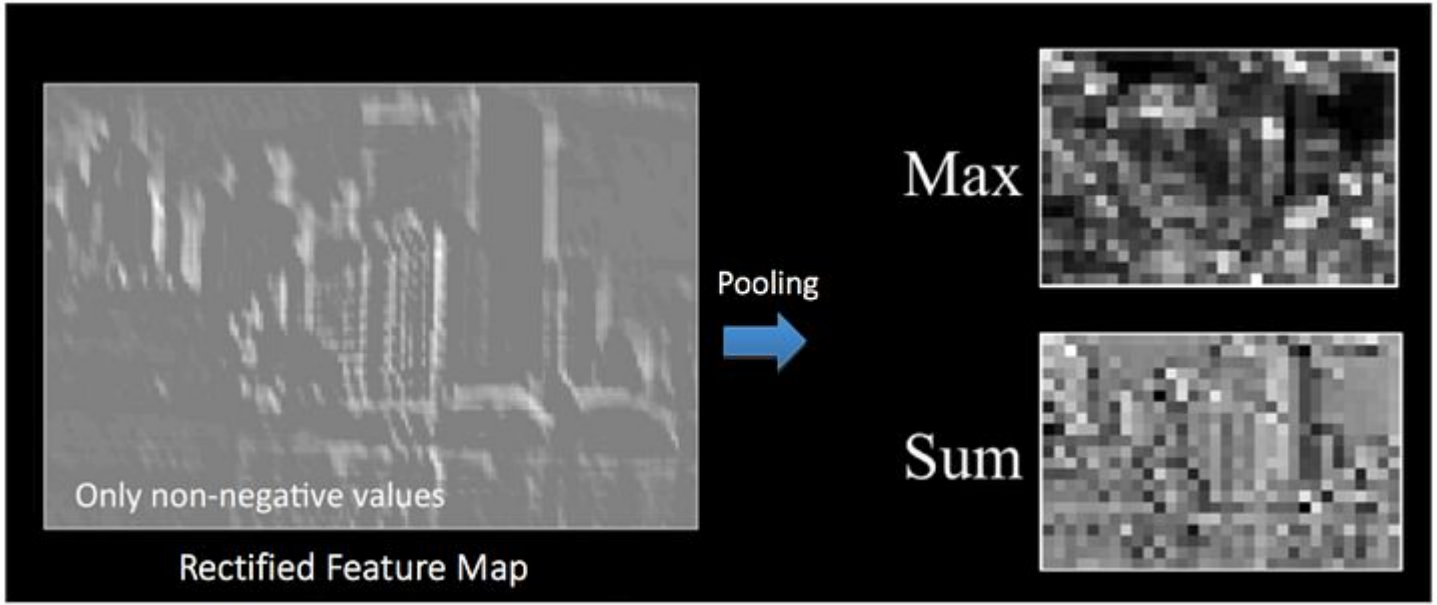
- reduce the spatial size of the representation
- reduce the (amount of parameters) computational complexity of the network
- makes network equivariant to small transformations, distortions, and translations in the input image (since we take the maximum/average value in a local neighborhood)

Types of pooling include: max, average, sum

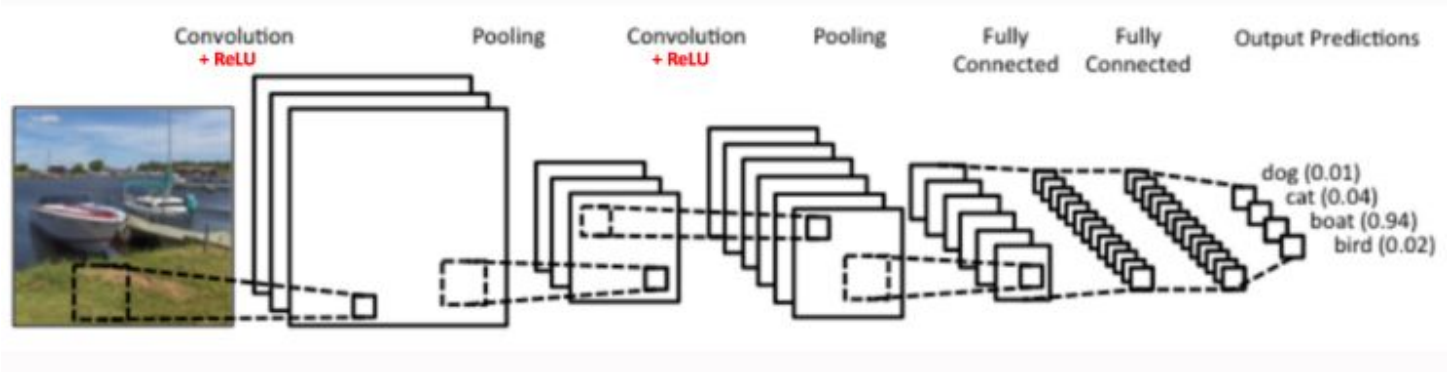




Pooling


Only non-negative values
Rectified Feature Map
Pooling
Max
Sum

Pooling


Convolution
+ ReLU
Pooling
Convolution
+ ReLU
Pooling
Fully Connected
Fully Connected
Output Predictions
dog (0.01)
cat (0.04)
boat (0.94)
bird (0.02)

Classification

Fully Connected Layer is a Multi Layer Perceptron (MLP) that uses an activation function in the output layer.

Flatten Layer converts a three-dimensional layer in the network into a one-dimensional vector to fit the input of a fully-connected layer for classification.

Choosing the right last-layer activation and loss function for your model

Problem type	Last-layer activation	Loss function
Binary classification	`sigmoid`	`binary_crossentropy`
Multiclass, single-label classification	`softmax`	`categorical_crossentropy`
Multiclass, multilabel classification	`sigmoid`	`binary_crossentropy`